Control de versions, documentació i integració continua

Custom GitHub Actions

Desplegament d'aplicacions Web
2 DAW

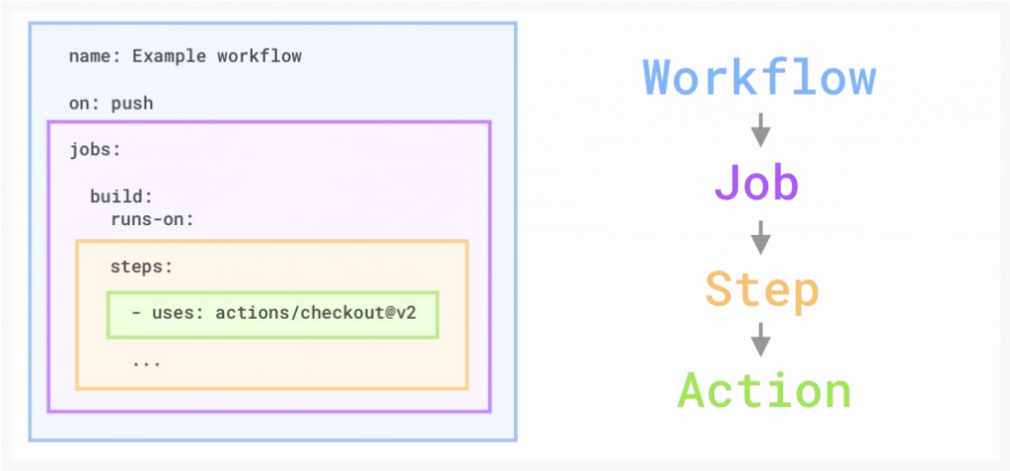
Índex

- Custom Github Actions
  - Introducció
  - Tipus possibles
  - Ubicar la nostra custom github action
  - Sintaxi
  - Exemple github Action en javascript

Custom Github Actions

Introducció

- Tasques individuals que pots combinar per crear feines i personalitzar el teu flux de treball.
- S'executen dins d'un step que forma part d'un job que compon un workflow

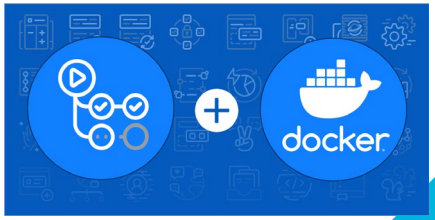


Custom Github Actions

Tipus possibles.

- Docker-container actions

  - Més consistent i confiable
  - No necessitem preocupar-nos per dependències o eines necessàries
  - Només es poden executar en entorn linux (runner)
  - Són més lentes



Custom Github Actions

Tipus possibles

- Javascript actions

  - Simplifica el codi de l'action
  - Execució més ràpida
  - Per garantir que es pot executar en tots els runners, cal que estiga implementada a Javascript pur.

Custom Github Actions

Sintaxis

- Les Actions requereixen un arxiu de metadades amb format YAML (.yml o .yaml).
- Les dades de l'arxiu de metadades defineixen les entrades, les eixides i el punt d'entrada principal per a l'Action.

https://docs.github.com/es/actions/creating-actions/metadata-syntax-for-github-actions

Custom Github Actions

Sintaxis

- Els principals elements que componen l'arxiu de metadades d'una Action són:

name (nom)
autor
descripción
inputs (entrades)
outputs (eixides)
outputs per a accions de passos d'execució compostos
runs per a accions de JavaScript
runs per a accions de passos d'execució compostos
runs per a accions de Docker
branding (marca)

```yaml
# action.yml
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:  # id of input
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time: # id of output
    description: 'The time we greeted you'
runs:
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
```

Custom Github Actions

Sintaxis

- name
  - **Requerit**
  - Especifica el nom de l'Action que GitHub mostra a la pestanya Actions

```yaml
# action.yml
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:  # id of input
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time: # id of output
    description: 'The time we greeted you'
runs:
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
```

Custom Github Actions

Sintaxis

- autor
  - **Opcional**
  - Nom de l'autor de l'Action
- description
  - **Requerit**
  - Descripció breu de l'Action

```yaml
# action.yml
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:  # id of input
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time: # id of output
    description: 'The time we greeted you'
runs:
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
```

Custom Github Actions

Sintaxis

- inputs
  - **Opcional**
  - Permet especificar paràmetres d'entrada que l'Action utilitzarà durant la seua execució

```yaml
# action.yml
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:  # id of input
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time: # id of output
    description: 'The time we greeted you'
runs:
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
```

Per cada input, GitHub crea una variable d'entorn per a l'entrada amb el nom INPUT_<VARIABLE_NAME>. La variable d'entorn creada converteix els nom d'entrada en lletres majúscula i reemplaça els espais amb els caràcters _.

Custom Github Actions

Sintaxis

- inputs

Tots els atributs d'un input són requerits a excepció del camp default que és opcional.

```yaml
# action.yml
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:  # id of input
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time: # id of output
    description: 'The time we greeted you'
runs:
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
```

Custom Github Actions

Sintaxis

- outputs
  - **Opcional**
  - Permet declarar dades que una Action estableix

Tots els atributs d'un output són requerits

```yaml
# action.yml
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:  # id of input
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time: # id of output
    description: 'The time we greeted you'
runs:
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
```

Custom Github Actions

Sintaxis

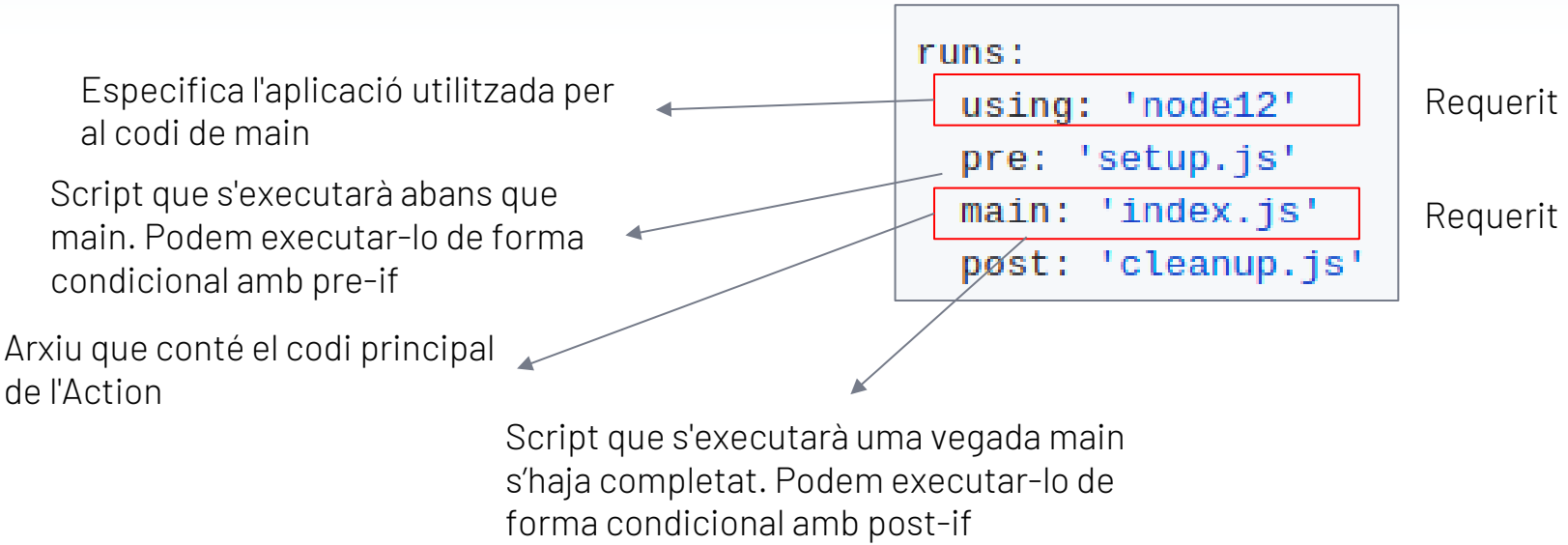
- runs
  - **Requerit**
  - La seua configuració difereix depenent de si és una Action de javascript o de Docker

```yaml
# action.yml
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:  # id of input
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time: # id of output
    description: 'The time we greeted you'
runs:
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
```

Custom Github Actions

Sintaxis

- runs (versión javascript)

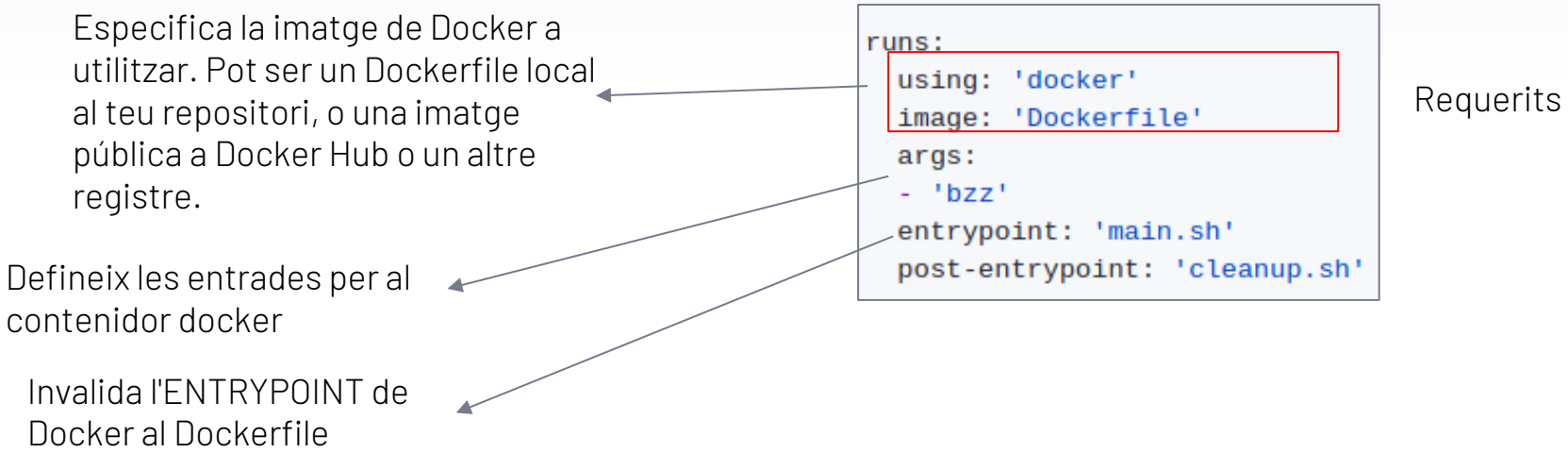


Custom Github Actions

Sintaxis

- runs (versión Docker)

  Especifica la imatge de Docker a utilitzar. Pot ser un Dockerfile local al teu repositori, o una imatge pública a Docker Hub o un altre registre.



Custom Github Actions

Sintaxis

https://feathericons.com/?query=award

- branding
  - Permet especificar un color i un icono per representar la nostra Action en cas de ser publicada al marketplace de github

```
branding:
  icon: 'award'
  color: 'green'
```

Custom Github Actions

Exemple en javascript

Passos:

- Crear fitxer action.yml
- Crear projecte de Node
  - Instal·lar llibreries necessàries
  - Crear index.js amb la lògica principal
  - Compilar codi
- Pujar canvis i provar l'action dins d'un workflow

Custom Github Actions

Exemple en javascript

Crear fitxer action.yml

- Ubiquem el .yml dins de .github/actions/hello-world-javascript-action/action.yml

```yaml
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:  # id of input
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time: # id of output
    description: 'The time we greeted you'
runs:
  using: 'node12'
  main: 'index.js'
```

Custom Github Actions

Exemple en javascript

- Crear projecte Node a la carpeta .github/actions/hello-world-javascript-action
  - npm init -y
- Instal·lar dependències necessàries del projecte

  npm install @actions/core -> (link)

  npm install @actions/github -> (link)

Custom Github Actions

Exemple en javascript

Crear index.js amb la lògica principal

```
const core = require('@actions/core');
const github = require('@actions/github');

try {
  // `who-to-greet` input defined in action metadata file
  const nameToGreet = core.getInput('who-to-greet');
  console.log(`Hello ${nameToGreet}!`);
  const time = (new Date()).toTimeString();
  core.setOutput("time", time);
  // Get the JSON webhook payload for the event that triggered the workflow
  const payload = JSON.stringify(github.context.payload, undefined, 2)
  console.log(`The event payload: ${payload}`);
} catch (error) {
  core.setFailed(error.message);
}
```

Custom Github Actions

Exemple en javascript

Compilar el projecte creat. Permetrà que no siga necessari pujar al nostre repositori el directori node_modules

- Instal·lar la dependència necessària per realitzar el compilat

  ```
  npm i -g @vercel/ncc
  ```

- Compilar l'arxiu principal

  ```
  ncc build index.js - -license licenses.txt
  ```

- Modifiquem el valor de main a l'arxiu action.yml

  ```
  main: 'dist/index.js'
  ```

Custom Github Actions

Exemple en javascript

Probar action en un workflow

```yaml
on: [push]

jobs:
  hello_world_job:
    runs-on: ubuntu-latest
    name: A job to say hello
    steps:
      # To use this repository's private action,
      # you must check out the repository
      - name: Checkout
        uses: actions/checkout@v2
      - name: Hello world action step
        uses: ./ # Uses an action in the root directory
        id: hello
        with:
          who-to-greet: 'Mona the Octocat'
      # Use the output from the `hello` step
      - name: Get the output time
        run: echo "The time was ${{ steps.hello.outputs.time }}"
```

Custom Github Actions

Exemple en Docker

Passos:

- Crear Dockerfile
- Crear fitxer d'action.yml
- Pujar canvis i provar l'action dins d'un workflow

Custom Github Actions

Exemple a Docker

Crear un Dockerfile

```
# Imagen del contenedor que ejecuta tu código
FROM alpine:3.10

# Copias tu archivo de código de tu repositorio
COPY entrypoint.sh /entrypoint.sh

# Archivo del código a ejecutar cuando comienza
ENTRYPOINT ["/entrypoint.sh"]
```

```
#!/bin/sh -l

echo "Hello $1"
time=$(date)
echo "::set-output name=time::$time"
```

Custom Github Actions

Exemple a Docker

Crear fitxer action.yml:

```yaml
# action.yml
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:  # id of input
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time: # id of output
    description: 'The time we greeted you'
runs:
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
```

Custom Github Actions

Exemple a Docker

Probar action en un workflow:

```yaml
on: [push]

jobs:
  hello_world_job:
    runs-on: ubuntu-latest
    name: A job to say hello
    steps:
      # To use this repository's private action,
      # you must check out the repository
      - name: Checkout
        uses: actions/checkout@v2
      - name: Hello world action step
        uses: ./ # Uses an action in the root directory
        id: hello
        with:
          who-to-greet: 'Mona the Octocat'
      # Use the output from the `hello` step
      - name: Get the output time
        run: echo "The time was ${{ steps.hello.outputs.time }}"
```